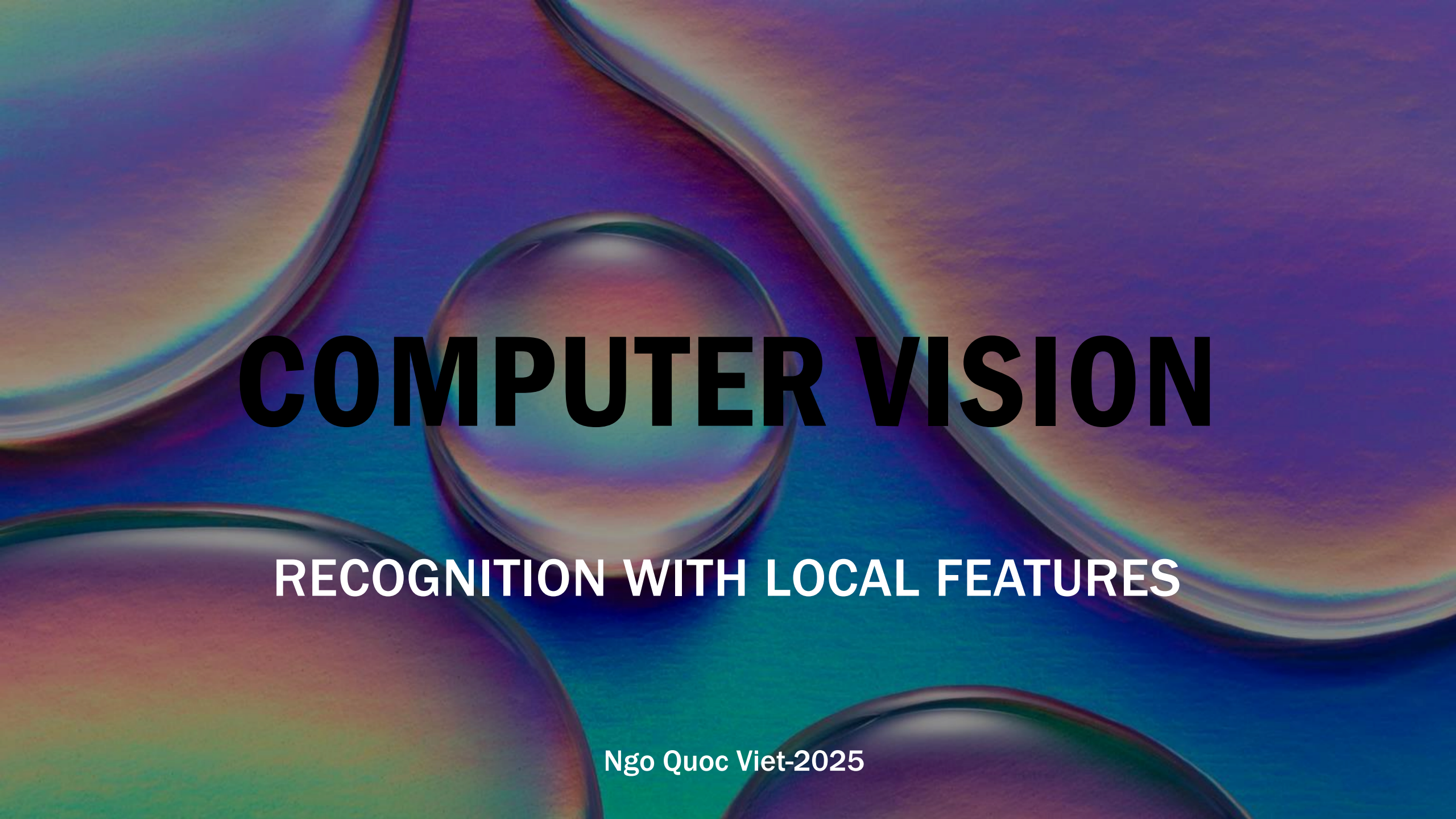




COMPUTER VISION

RECOGNITION WITH LOCAL FEATURES

Ngo Quoc Viet-2025

CONTENTS

1. Detector and Descriptor for local features

- Invariant Local Features
- Keypoint Detector
- Local feature Detector and Descriptor

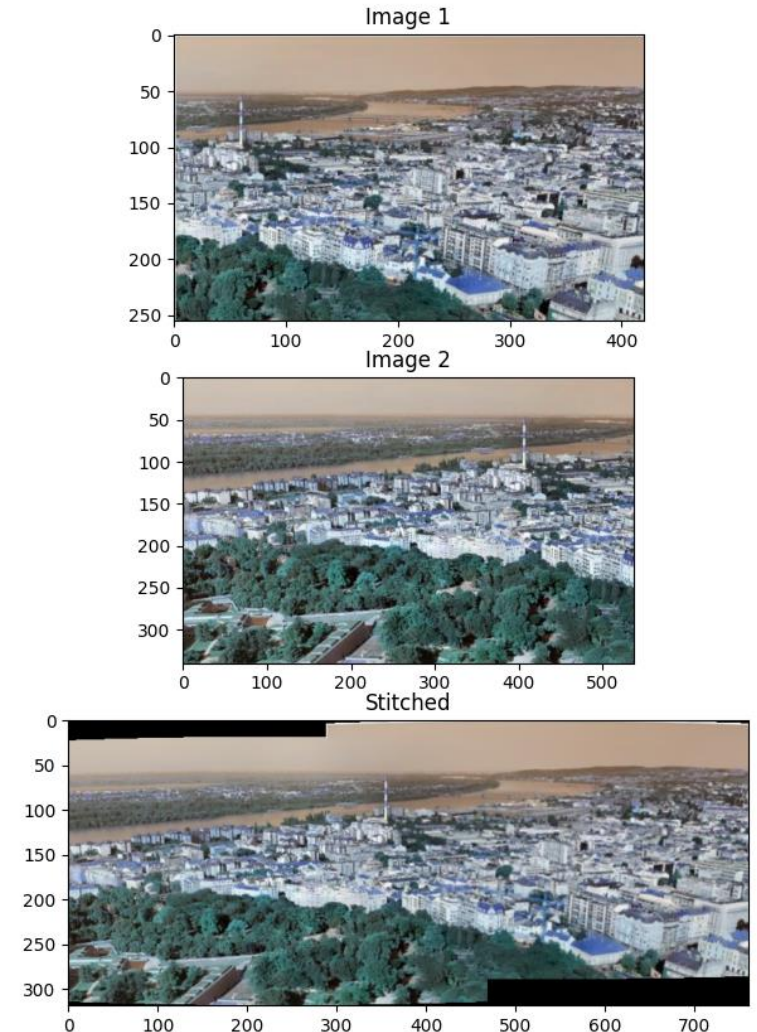
2. Recognition using local features

- Local Feature Matching
- Affine and Homography Estimation

3. Panorama

LECTURE OUTCOMES

- Understanding Detector and Descriptor for local features.
- SIFT
- Affine and Homography Estimation
- Local Feature Matching
- Implementing panorama project using SIFT and homography estimation



LOCAL FEATURES

- Obtain a representation that allows us to find a particular object we've encountered before
- Local features based on the appearance of the object at particular interest points.
- Features should be reasonably invariant to illumination changes, and ideally, also to scaling, rotation, and minor changes in viewing direction
- In addition, we can use local features for matching, this is useful for tracking and 3D scene reconstruction.

LOCAL FEATURES

Key properties of a good local feature:

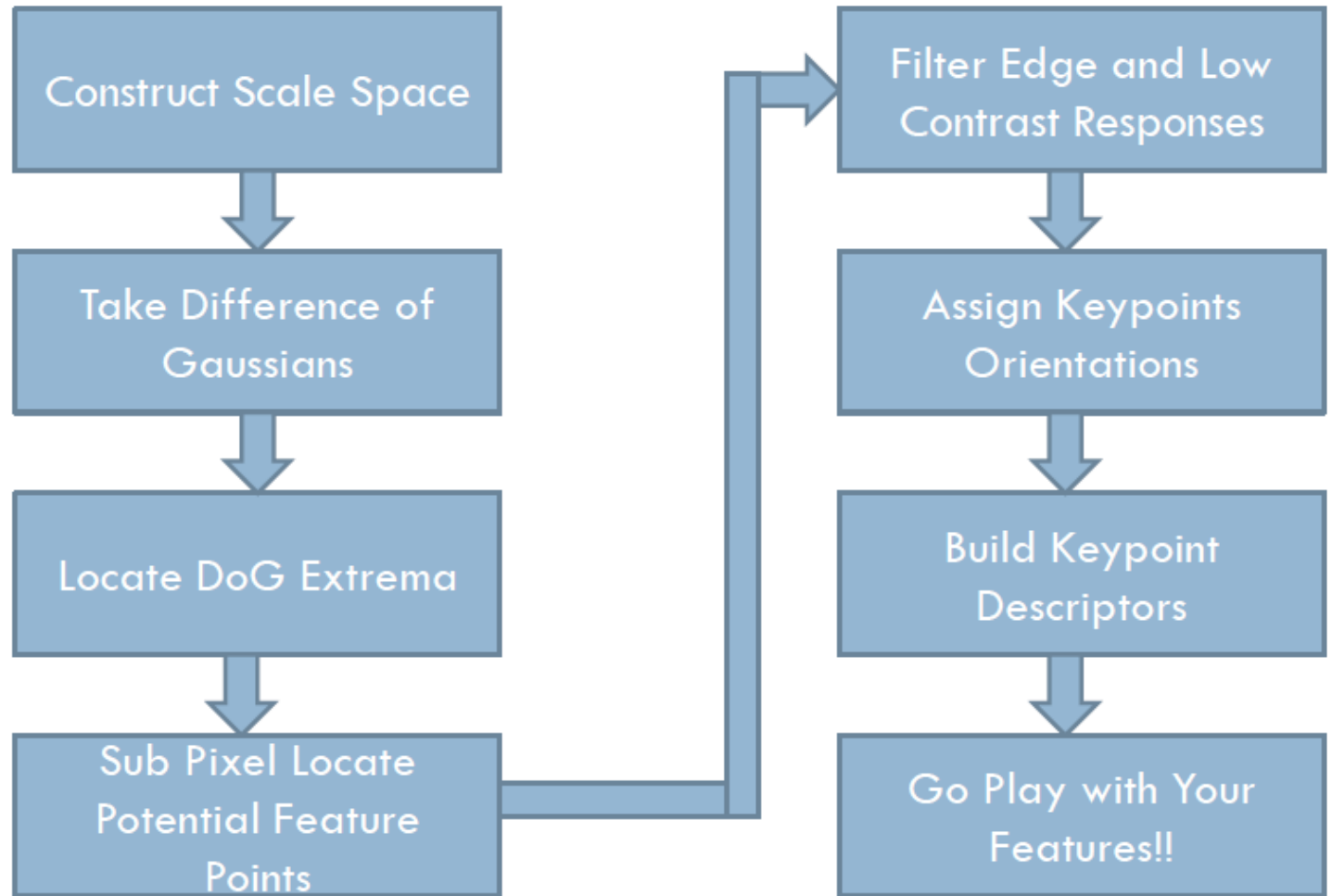
- Must be highly distinctive, a good feature should allow for correct object identification with low probability of mismatch.
- Should be easy to extract.
- Invariance, a good local feature should be tolerant to
 - Image noise
 - Changes in illumination
 - Uniform scaling
 - Rotation
- Should be easy to match against a (large) database of local features.

SCALE INVARIANT FEATURE TRANSFORM

- Scale Invariant Feature Transform (SIFT) is an approach for detecting and extracting local feature descriptors that are reasonably invariant to changes in illumination, image noise, rotation, scaling, and small changes in viewpoint.
- It detects distinctive key points or features in an image that are robust to changes in scale, rotation, and affine transformations.
- SIFT works by identifying keypoints based on their local intensity extrema and computing descriptors that capture the local image information around those keypoints.
- These descriptors can then be used for tasks like image matching, object recognition, and image retrieval.
- The major advantage of SIFT features, over-edge features, or hog features is that they are not affected by the size or orientation of the image.

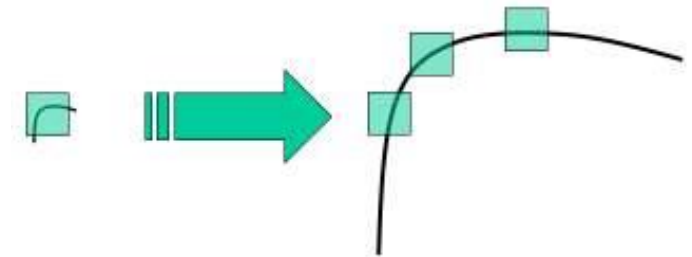
SIFT STAGES

- Detection stages for SIFT features:
- Scale-space extrema detection.
- Keypoint localization.
- Orientation assignment.
- Generation of keypoint descriptors.



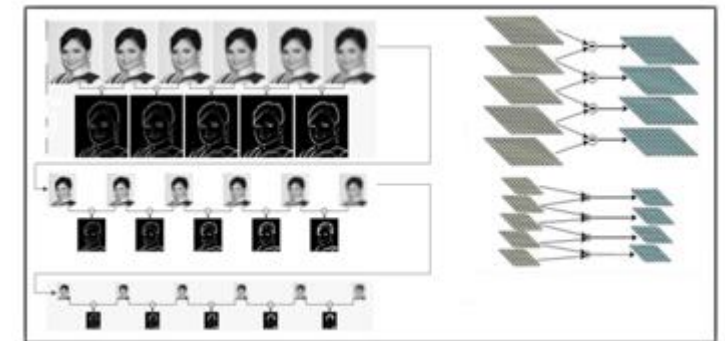
SCALE-SPACE EXTREMA DETECTION

- A corner may not be a corner if the image is scaled. A corner in a small image within a small window is flat when it is zoomed in the same window. We can't use the same window to detect keypoints with different scale. That's why we need to build scale-space.
- We need to identify the most distinct features in a given input image while ignoring any noise as well as ensure that the features are not scale-dependent.
- To create a scale space, you take the original image and generate progressively blurred out images.



Pic from:

https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html



Pic from:

<https://www.astesj.com/v06/i01/p143/>

SCALE-SPACE EXTREMA DETECTION

- Gaussian kernel is used to create scale space

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

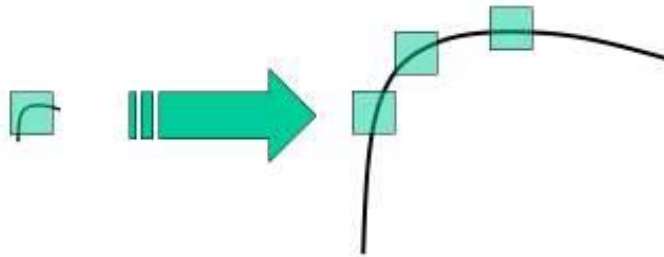
where

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

- σ is the “scale” parameter. Consider it as amount of blurring.
- Amount of blurring
- Assume the amount of blur in a particular image is σ . Then, the amount of blur in the next image will be $k * \sigma$.

SCALE-SPACE EXTREMA DETECTION

- Laplacian of Gaussian ($LoG = \sigma^2 \Delta^2 G$) filtering is used to build Scale-space. LoG acts as a blob detector which detects blobs in various sizes due to change in σ .
- In short, σ acts as a scaling parameter. Gaussian filter with low σ gives high value for small corner while gaussian kernel with high σ fits well for larger corner.



- But this LoG is a little costly, so SIFT algorithm uses Difference of Gaussians (DoG) which is an approximation of LoG.

- Gaussian filter with scale σ is determined by.

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

- LoG filter with scale σ is determined by.

$$LoG_{\sigma} = -\frac{1}{\pi\sigma^4} \left(1 - \frac{x^2 + y^2}{2\sigma^2} \right) e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

- DoG filter with two different σ_1 and σ_2 ($\sigma_2 > \sigma_1$) is determined by.

$$DoG_{\sigma_1, \sigma_2} = \frac{1}{2\pi} \left[\frac{1}{\sigma_1^2} e^{-\frac{(x^2+y^2)}{2\sigma_1^2}} - \frac{1}{\sigma_2^2} e^{-\frac{(x^2+y^2)}{2\sigma_2^2}} \right]$$

- In SIFT, $\sigma_2 = k\sigma_1$.

```
img = cv2.GaussianBlur(img, (5, 5), sigma)
```

```
img = cv2.Laplacian(img, cv2.CV_64F)
```

SCALE-SPACE EXTREMA DETECTION

- Approximation of Laplacian of Gaussians

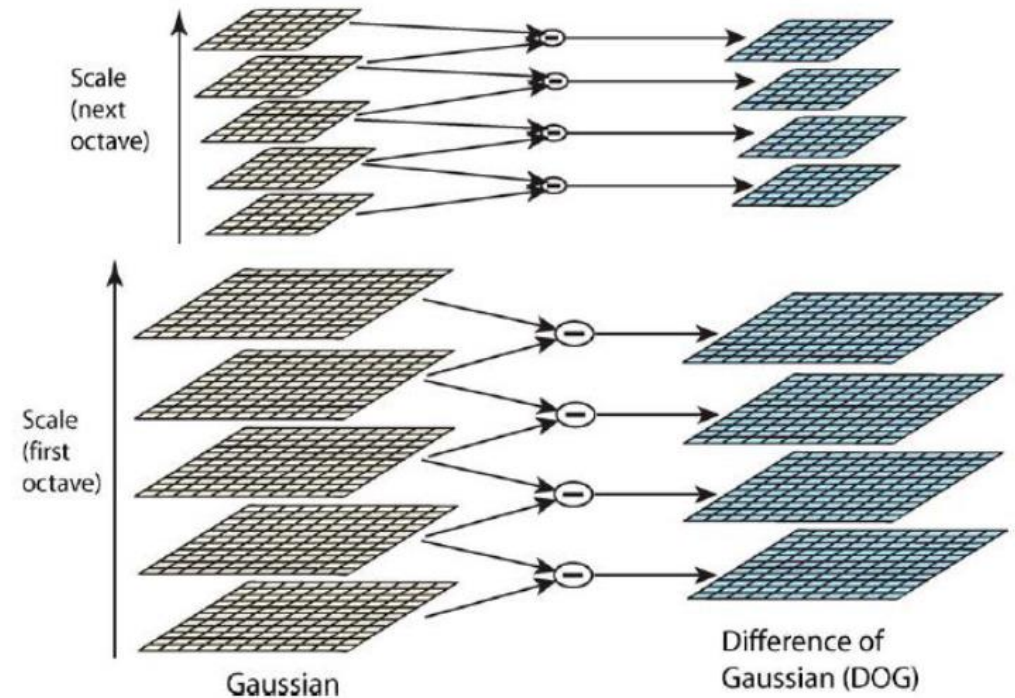
$$\sigma \nabla^2 G = \frac{\partial G}{\partial \sigma} \approx \frac{G(x, y, k\sigma) - G(x, y, \sigma)}{k\sigma - \sigma}$$

$$\begin{aligned} & \frac{G(x, y, k\sigma) - G(x, y, \sigma)}{k\sigma - \sigma} \\ & \approx (k - 1)\sigma^2 \nabla^2 G \end{aligned}$$

$$\begin{aligned} & D(x, y, \sigma) \\ & = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) \end{aligned}$$

$$D(x, y, \sigma) = (L(x, y, k\sigma) - L(x, y, \sigma))$$

- The difference of the Gaussian-blurred images at scales σ and $k\sigma$.



For each octave of scale space, initial image is repeatedly convolved with Gaussians to produce the set of scale space images. There are some fixed levels in each octave. After each octave, the Gaussian image is down-sampled by factor of 2, and the process is repeated.

SCALE-SPACE EXTREMA DETECTION

- Just the Laplacian of Gaussian images aren't great. They are not scale invariant. See the σ^2 at denominator of $G(x, y, \sigma)$.

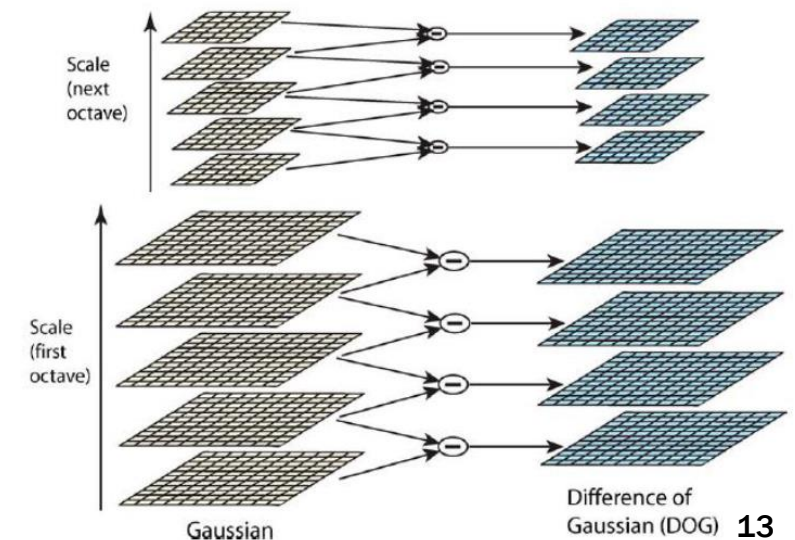
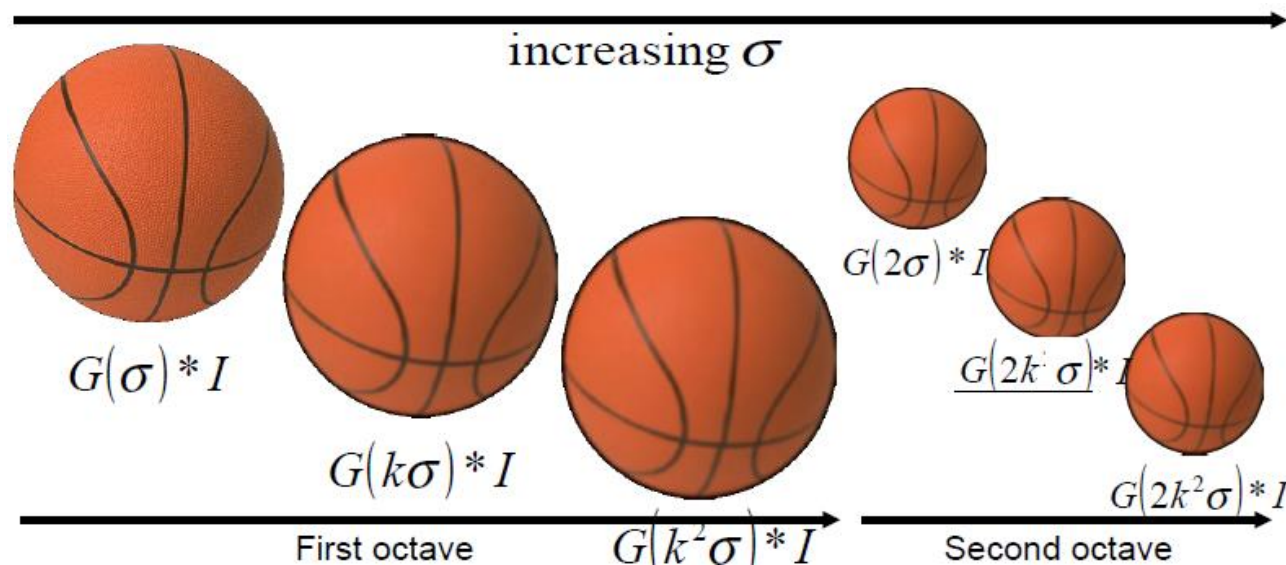
- Then the scale invariant Laplacian of Gaussian would look like this

$$\sigma^2 \Delta^2 G$$

- The resultant images after the DoG operation are already multiplied by the $\sigma^2 \rightarrow$ **Scale Invariant**.
- However, we still have $(k-1)$ in DoG pyramid.
- But we'll just be looking for the *location* of the maximums and minimums in the images.

SCALE-SPACE EXTREMA DETECTION

- The first step toward the detection of interest points is the convolution of the image with Gaussian filters at different scales, and the generation of difference of Gaussian from the difference of adjacent blurred images
- The convolved images are grouped by octave (an octave corresponds to doubling the value of σ)
- And the value of k is selected so that we obtain a fixed number of blurred images per octave. If $k=1$, each octave has only one DoG image.



SCALE-SPACE EXTREMA DETECTION

- The domain of the variable σ is discretized in logarithm steps arranged in O octaves.
- Each octave is further subdivided in S sublevels.
- The octave index o and sublevel index s are mapped to corresponding scale σ by the formula

$$\sigma(o, s) = \sigma_0 2^{o+s/S}, o \in o_{min} + [0, \dots, O - 1], s \in [0, S - 1]$$

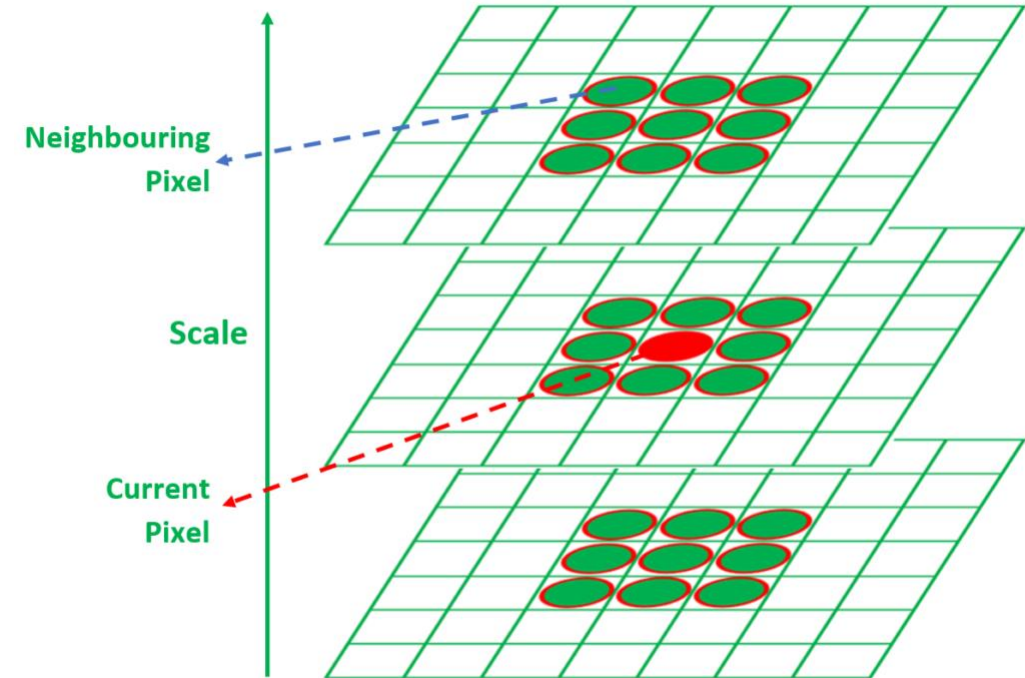
- Scale multiplicative step: $k = 2^{1/S}$
- If there is no σ_0 then: $\sigma_0 = 1.6 * k$
- Scale step factor: $d\sigma_0 = \sigma_0 \cdot \sqrt{1 - 1/k^2}$

SCALE-SPACE EXTREMA DETECTION

- **Octaves and Scales**
- The number of octaves and scale depends on the size of the original image. The creator of SIFT suggests that 4 octaves and 5 blur levels are ideal for the algorithm
- **The first octave**
- If the original image is doubled in size and antialiased a bit (by blurring it) then the algorithm produces more four times more keypoints. The more the keypoints, the better.

KEYPOINT LOCALIZATION

- Keypoint Localization by locating DoG Extreme
- Keypoints are maxima (minima) of DoG that occur at multiple scales.
- Locate detector via DoG extreme of all scales
- Choose all extreme within 3x3x3 neighborhood. To locate the local maxima and minima, we go through every pixel in the image and compare it with its neighboring pixels (DoG image and 2 adjacent images (26 comparisons) to find Min and Max → **candidate keypoints**).



KEYPOINT LOCALIZATION

- Once potential keypoints locations are found, they have to be refined to get more accurate results.
- Hence, we will eliminate the keypoints that have low contrast or lie very close to the edge.
- To deal with the low contrast keypoints, a **second-order Taylor expansion** is computed for each keypoint. If the resulting value is less than 0.03 (in magnitude), this keypoint is rejected.
- These are the keypoints that are close to the edge and have a high edge response but may not be robust to a small amount of noise (edges also need to be removed). **A second-order Hessian matrix** is used to identify such keypoints.
- We will now assign an **orientation value for each keypoint** to make the rotation invariant.

KEYPOINT LOCALIZATION

- Taylor series expansion of degree two of scale-space function, $D(x, y, \sigma)$, shifted so that the origin is at the candidate keypoint. D represents the Difference of Gaussian.

$$D(\vec{x}) = D + \left(\frac{\partial D}{\partial \vec{x}} \right)^T \vec{x} + \frac{1}{2} \vec{x}^T \left(\frac{\partial^2 D}{\partial \vec{x}^2} \right) \vec{x}; \vec{x} = (x, y, \sigma)^T$$

$(x, y, \sigma)^T$ is the **offset** from candidate keypoint.

- The location of the extrema is determined by taking derivative of this function with respect to $\vec{x}$ and setting it to zero (differentiate and set to 0).
- Minimize to get true location maxima

$$\hat{x} = - \left(\frac{\partial^2 D}{\partial \vec{x}^2} \right)^{-1} \times \frac{\partial D}{\partial \vec{x}}$$

- To remove the unstable key points, the value of $\hat{x}$ is calculated and if the function value at $\hat{x}$ is below a threshold value then the point is excluded

KEYPOINT LOCALIZATION-CONTRAST THRESHOLD

- Let $\vec{a}$ be a candidate keypoint, $\vec{x}$ are neighbors of $\vec{a}$. Taylor expansive of degree two is determine by

$$D(\vec{x}) = D(\vec{a}) + D'(\vec{a})(\vec{x} - \vec{a}) + \frac{1}{2}D''(\vec{a})(\vec{x} - \vec{a})^2$$

- Extreme is the location which $D'(\vec{x}) = 0$. In this case, $D'(\vec{a}) + D''(\vec{a})(\vec{x} - \vec{a}) = 0$

$$\hat{x} = -\frac{D'(\vec{a})}{D''(\vec{a})} + \vec{a}$$

- Low Contrast Points Filter using Scale Space values at candidate keypoints. Substituting $\hat{x}$ from above formula to $D(\vec{x})$, we have

$$D(\hat{x}) = D + \frac{1}{2} \left(\frac{\partial D}{\partial x} \right)^{-1} \hat{x}$$

- Remove keypoints which have $|D(\hat{x})|$ smaller than 0.03 (images values in $[0, 1]$).

KEYPOINT LOCALIZATION-EDGE THRESHOLD

- The idea is to calculate two gradients at the keypoint. Both perpendicular to each other. Based on the image around the keypoint, three possibilities exist. The image around the keypoint can be
 - **A flat region:** both gradients will be small
 - **An edge:** one gradient will be big (perpendicular to the edge) and the other will be small (along the edge)
 - **A “corner”:** both gradients will be big. Corners are great keypoints. If both gradients are big enough, we let it pass as a key point. Otherwise, it is rejected.
- Mathematically, this is achieved by the Hessian Matrix. More detail:
<http://www.aishack.in/2010/04/harris-corner-detector/>

ORIENTATION ASSIGNMENT

- Use scale of point (scale at key point is determined) to choose correct image.

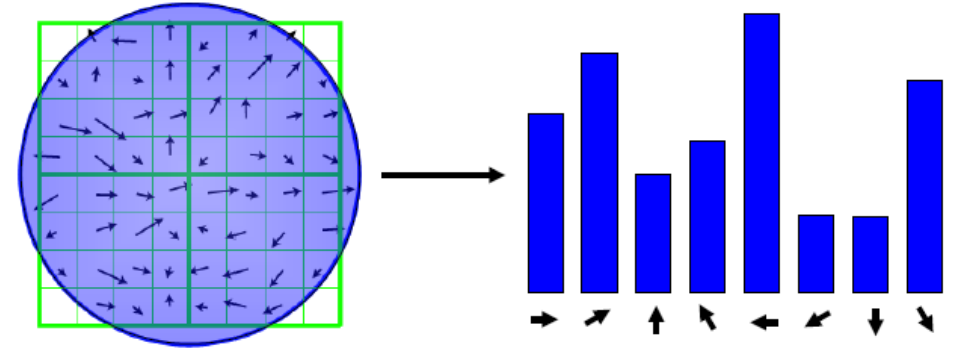
$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

- Compute gradient magnitude and orientation

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1} \left(\frac{L(x, y+1) - L(x, y-1)}{L(x+1, y) - L(x-1, y)} \right)$$

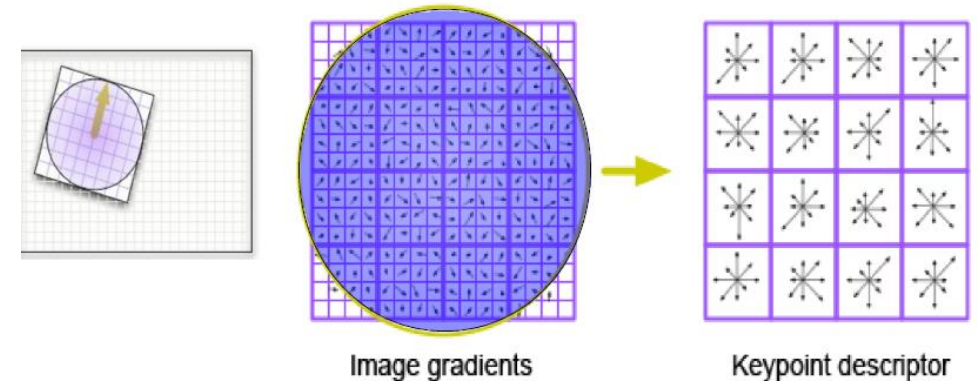
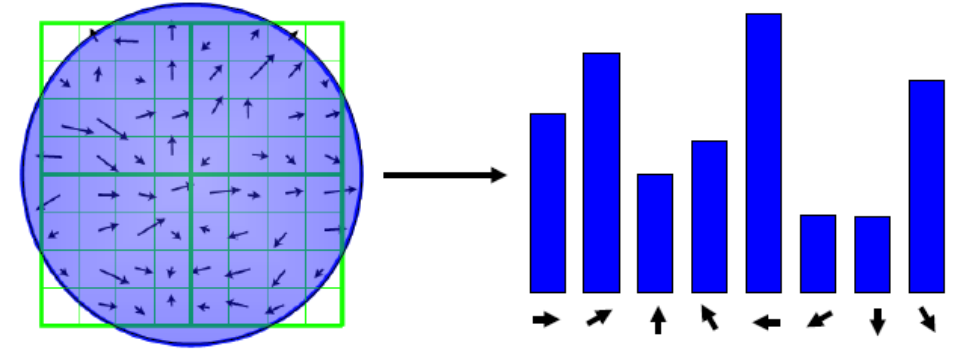
- Create gradient histogram (36 bins – 10 degrees for each)
- Weighted by magnitude and Gaussian window (σ is 1.5 times that of the scale of keypoint).
- Calculate orientation for every point Gaussian window, then create histogram for them



To achieve detection which is invariant with respect to the rotation of the image, orientation needs to be calculated for the key-points. This is done by considering the neighborhood of the keypoint and calculating the magnitude and direction of gradients of the neighborhood. Based on the values obtained, a histogram is constructed with 36 bins to represent 360 degrees of orientation (10 degrees per bin).

KEYPOINT DESCRIPTOR

- Each keypoint has $(x, y, \sigma, m, \theta)$.
- A 16×16 neighborhood of the keypoint is used for defining the descriptor of that key-point. This 16×16 neighborhood is divided into sub-block. Each such sub-block is a non-overlapping, contiguous, 4×4 neighborhood.
- 4×4 Gradient window (4×4 pixel sub-regions)
- Histogram of 4×4 samples per window in 8 directions
- $4 \times 4 \times 8 = 128$ dimension feature vector (4×4 histograms, 8 bins for each). We can use 2×2 histograms or else.



SIFT DESCRIPTOR

- In order to get orientation invariance, the coordinates of the descriptor and gradients orientations are **rotated relative to the keypoint orientation**.
- The purpose of “ σ equal 1.5 times of σ of keypoint scale” is to avoid sudden changes in the descriptor with small changes in the position of the window, and to give less emphasis to gradients that are far from the center of the descriptor.
- Finally, feature vector is normalized to unit to avoid linear changes of illumination.

SIFT IN OPENCV

```
3  img = cv2.imread('../datasets/others/box_in_scene.png')
4  # Converting image to grayscale
5  gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6  # Applying SIFT detector
7  sift = cv2.SIFT_create()
8  sift.setContrastThreshold(0.03)
9  sift.setEdgeThreshold(5)
10 # Detect key points and compute descriptors
11 keypoints, descriptors = sift.detectAndCompute(gray, None)
12 for x in keypoints:
13     print("{:.2f},{:.2f}) = size {:.2f} angle {:.2f}".format(x.pt[0], x.pt[1], x.size, x.angle))
14
15 kp = sift.detect(gray, None)
16 # Marking the keypoint on the image using circles
17 img = cv2.drawKeypoints(gray,
18                          keypoints,
19                          img,
20                          flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
21 cv2.imshow('SIFT', img)
22 cv2.waitKey(0)
23 cv2.destroyAllWindows()
```

MATCHING WITH SIFT

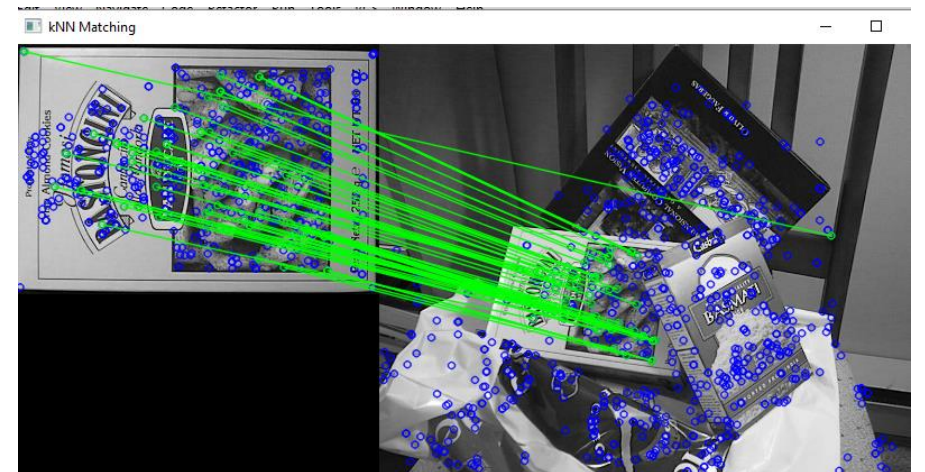
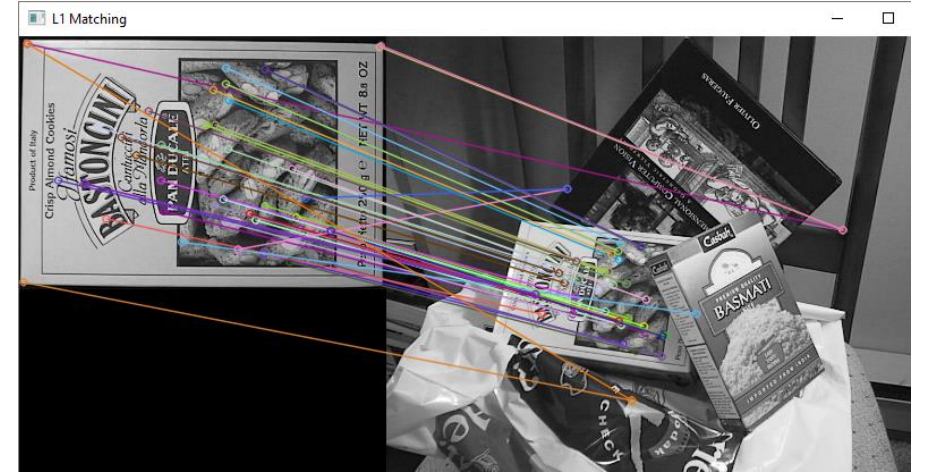
- **Brute force Matcher** with L_1 or L_2 or kNN to match two SIFT descriptors.
- Using `bf = cv2.BFMatcher` → `bf.match` with L_1 or L_2 distances.
- Using `bf.knnMatch` with Ratio Test
- **FLANN based Matcher:** FLANN stands for Fast Library for Approximate Nearest Neighbors. It contains a collection of algorithms optimized for fast nearest neighbor search in large datasets and for high dimensional features. It works faster than BFMatcher for large datasets.
- For FLANN based matcher, we need to **pass two dictionaries** which specifies the algorithm to be used, its related parameters etc

MATCHING WITH SIFT

```
#feature matching
bf = cv2.BFMatcher(cv2.NORM_L1, crossCheck=False)
matches = bf.match(descriptors_1, descriptors_2)
matches = sorted(matches, key = lambda x:x.distance)
img3 = cv2.drawMatches(img1, keypoints_1, img2, keypoints_2, matches[:50], img2, flags=2)

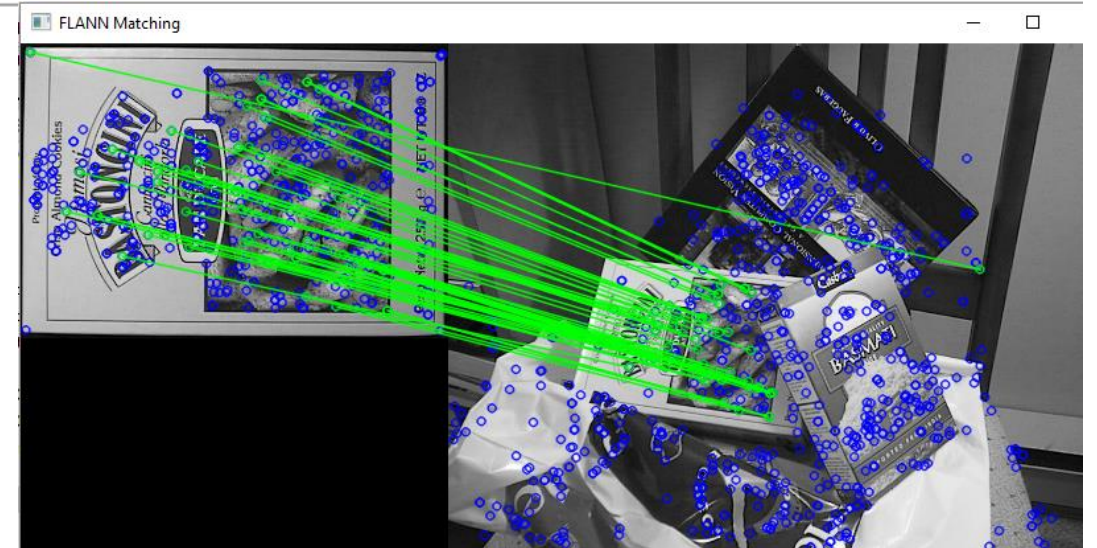
matches = bf.knnMatch(descriptors_1, descriptors_2, k = 2)
```

```
# Apply ratio test
good = []
for m,n in matches:
    if m.distance < 0.7*n.distance:
        good.append([m])
img4 = cv2.drawMatchesKnn(img1, keypoints_1, img2, keypoints_2, good, None,
                           matchColor=(0, 255, 0), matchesMask=None,
                           singlePointColor=(255, 0, 0), flags=0)
```



MATCHING WITH SIFT

```
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks=50) # or pass empty dictionary
flann = cv2.FlannBasedMatcher(index_params,search_params)
flann = cv2.FlannBasedMatcher(index_params,search_params)
matches = flann.knnMatch(descriptors_1,descriptors_2,k=2)
# Need to draw only good matches, so create a mask
matchesMask = [[0,0] for i in range(len(matches))]
# ratio test as per Lowe's paper
for i,(m,n) in enumerate(matches):
    if m.distance < 0.7*n.distance:
        matchesMask[i]=[1,0]
draw_params = dict(matchColor = (0,255,0),
                    singlePointColor = (255,0,0),
                    matchesMask = matchesMask,
                    flags = cv2.DrawMatchesFlags_DEFAULT)
img5 = cv2.drawMatchesKnn(img1,keypoints_1,img2,keypoints_2,matches,None,**draw_params)
```

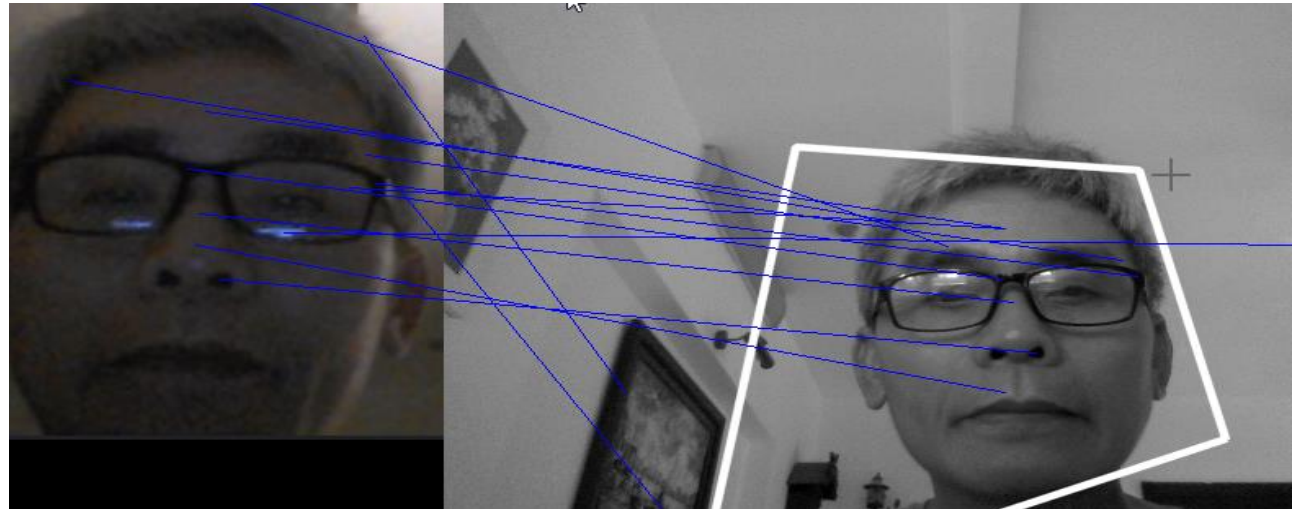


FACE DETECTION WITH SIFT

- Extract SIFT descriptors of source image (face being detected) and target (video frames) as usual.
- Matching two SIFT descriptors with *cv2.BFMatcher* or *cv2.FlannBasedMatcher*.
- Check if number of matched descriptors is greater threshold.

```
while cap.isOpened():
    _, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    keypoints_2, descriptors_2 = sift.detectAndCompute(gray, None)
    matches = flann.knnMatch(descriptors_1, descriptors_2, k=2)
    good = []
    for m, n in matches:
        if m.distance <= 0.9 * n.distance:
            good.append([m])

    img3 = cv2.drawMatchesKnn(img, keypoints_1, gray, keypoints_2, good, None,
                              matchColor=(0, 255, 0), matchesMask=None,
                              singlePointColor=(255, 0, 0), flags=0)
    cv2.imshow('Flann Match', img3)
    if len(good) >= 10:
        src_pts = np.float32([keypoints_1[m[0].queryIdx].pt for m in good]).reshape(-1, 1, 2)
        dst_pts = np.float32([keypoints_2[m[0].trainIdx].pt for m in good]).reshape(-1, 1, 2)
```



AFFINE TRANSFORMATION ESTIMATION

- Affine estimation, similar to homography estimation, is a concept in computer vision and image processing.
- Affine transformation estimation is the process of estimating the parameters of an affine transformation based on point data.
- An affine transformation includes operations such as translation, rotation, scaling, and projection.
- Affine transformation estimation is a crucial tool in computer vision. It is used in: Image Registration (aligning images from different sources or capturing conditions), Object Recognition (Align and normalize object instances for recognition. When objects undergo translation, rotation, scaling, or shearing, estimating the affine transformation allows for invariant representation); Panorama Stitching (creating panoramic images from multiple photos, each photo needs to be aligned properly to form a seamless panorama)

AFFINE TRANSFORMATION ESTIMATION

- **Prepare Corresponding Points:** Identify corresponding points between the source space and the target space
- **Construct Data Matrix:** Create a data matrix that holds all the corresponding points from both spaces. Each row of the matrix represents a pair of corresponding points.
- **Setup the System of Equations:**
- **Affine transformation matrix for a 2D space and 3D:**
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
- **Solve the System of Equations:** Use methods like the least squares fitting or RANSAC to solve the system of equations and estimate the parameters of the affine transformation. The solution should minimize the difference between the actual and transformed coordinates. These parameters include values for performing translation, rotation, scaling, and projection

AFFINE TRANSFORMATION ESTIMATION

- Estimating an affine transformation typically involves finding the coefficients of the transformation matrix that maps points from one coordinate system to another.
- Establish point correspondences (at least 4 points in the input image and 4 corresponding points in the target image).
- Various techniques can be used to estimate affine transformations, including direct linear transformation (DLT), least squares methods, and robust estimation techniques like RANSAC (Random Sample Consensus).

- `cv2.estimateAffine2D(srcPts, dstPts, cv2.RANSAC, ransacReprojThreshold=5)`
`ransacReprojThreshold`: Maximum allowed reprojection error to treat a point pair as an inlier (used in the RANSAC and RHO methods only).

HOMOGRAPHY ESTIMATION

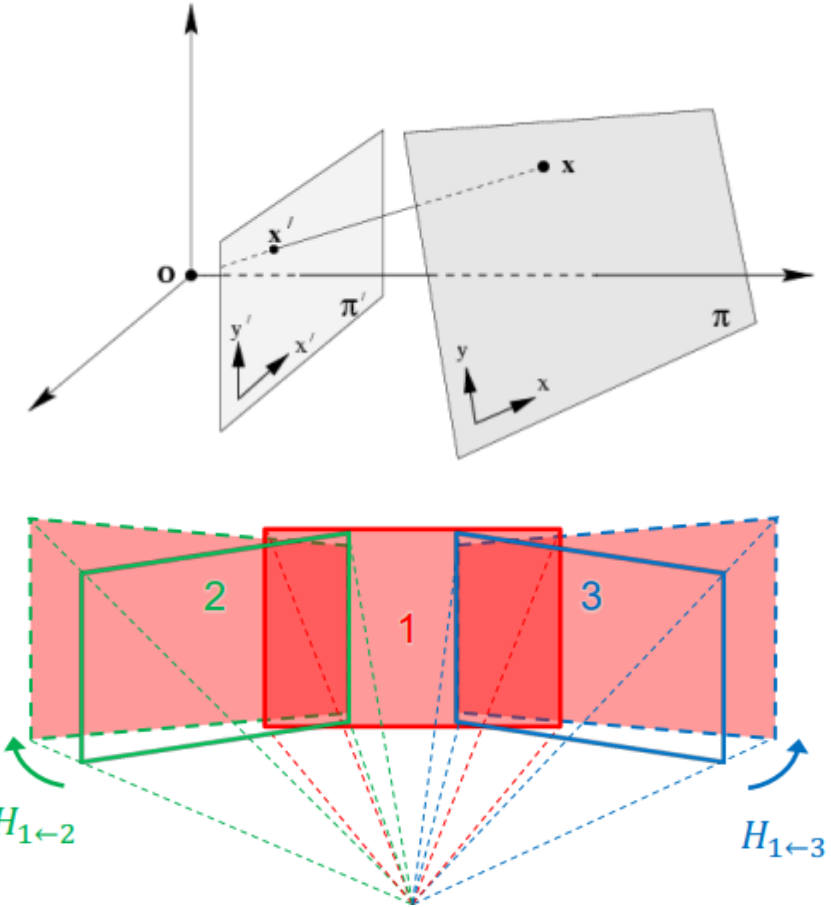
- Homography estimation is a technique used in computer vision and image processing to find the relationship between two images of the same scene, but captured from different viewpoints.
- A homography is a transformation that maps points from one plane to another. In the context of computer vision, it is often used to represent the relationship between two images of the same scene taken from different viewpoints or under different camera poses.
- A homography matrix (H) is a 3×3 matrix that defines a projective transformation between two planes. Given a set of corresponding points in the two images, the goal of homography estimation is to compute the matrix H that best describes the transformation between the images. This is particularly useful in applications such as image stitching, object recognition, and augmented reality.
- The homography estimation process involves solving a system of linear equations formed by the correspondences between points in the two images. Methods like the **Direct Linear Transform (DLT)** or **RANSAC (Random Sample Consensus)** are commonly used for this purpose.

HOMOGRAPHIES INDUCED BY CENTRAL PROJECTION

- Projection along rays through a common point (center of projection) defines a mapping from one plane to another.
- Central projection maps points on one plane to points on another plane.
- Projection also maps lines to lines: consider a plane through projection center that intersects the two planes → lines mapped onto lines.
- Central projection may be expressed by

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

in which, s is scale factor



ESTIMATING THE HOMOGRAPHY BETWEEN OVERLAPPING IMAGES

- Establish point correspondences (at least 4 points in the input image and 4 corresponding points in the target image). **The problem is that there are a lot of corresponding points between 2 images → how to select best-fit 4 point pairs.** RANSAC (Random Sample Consensus).
- RANSAC is a learning technique to estimate parameters of a model by random sampling of observed data. In this case, RANSAC samples any 4 pairs of random points and calculates matrix H, calculates the loss between target points and input points after transforming with H. Loss is determined by

$$loss = \sum_i distance(H * x_i, x'_i)$$

in which, (x_i, x'_i) is point pair. Repeating the sampling process – calculate this loss a large enough number of times, then choose H with the minimum loss.

- More RANSAC at: <https://www.cse.psu.edu/~rtc12/CSE486/lecture15.pdf>, <https://www.baeldung.com/cs/ransac>.

- `cv2.findHomography(srcPts, dstPts, cv2.RANSAC, ransacReprojThreshold=5)`

***`ransacReprojThreshold`**: Maximum allowed reprojection error to treat a point pair as an inlier (used in the RANSAC and RHO methods only).*

AFFINE ESTIMATION VS. HOMOGRAPHY ESTIMATION

	Affine	Homography
Parameters	6 for 2D or 12 for 3D	8 for 2D or 16 for 3D
Use cases	Affine transformations are suitable for applications where perspective changes are negligible, such as image resizing, object recognition, and texture mapping in computer graphics	Homography is often used when dealing with images taken from different viewpoints or perspectives. It is suitable for image stitching, panorama creation, or camera calibration
Perspective Distortions	Affine transformations cannot model perspective distortions accurately. While they can handle scaling, rotation, and translation	Homography can handle perspective distortions, which occur when objects are viewed from different angles or distances
Matrix size	2x3 for 2D and 3x4 for 3D	3x3 for 2D and a 4x4 for 3D
Correspondence Points	Affine transformation estimation requires three or more corresponding points between two images or scenes	Homography estimation typically requires four or more corresponding points between two images or scenes

PANORAMA IMAGE

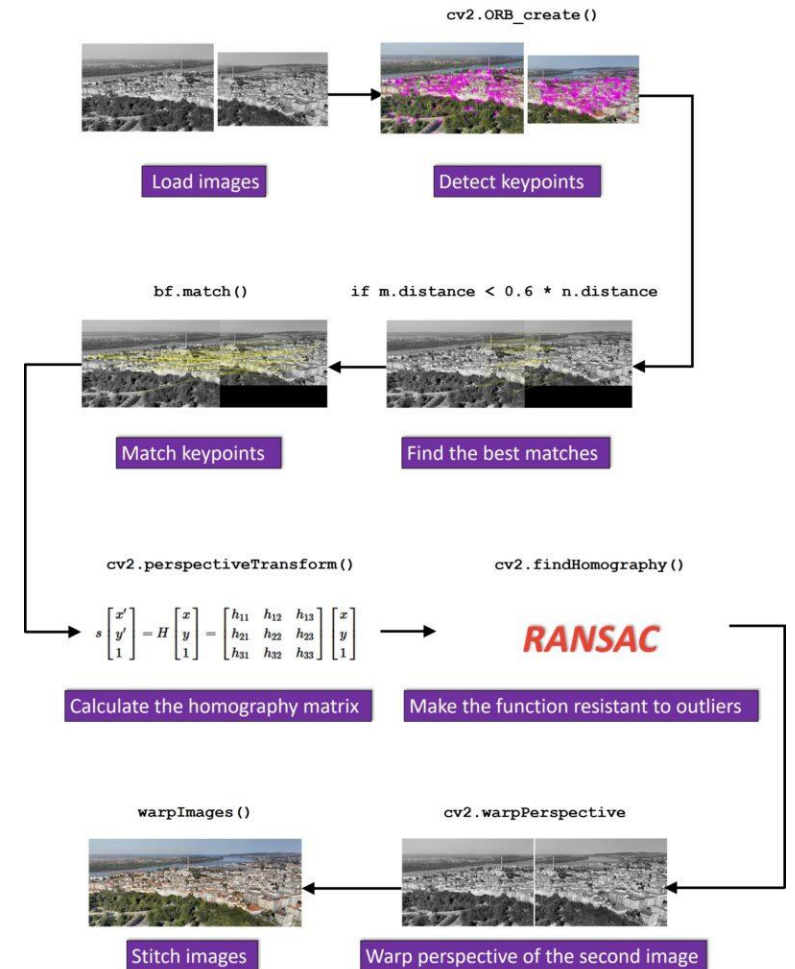
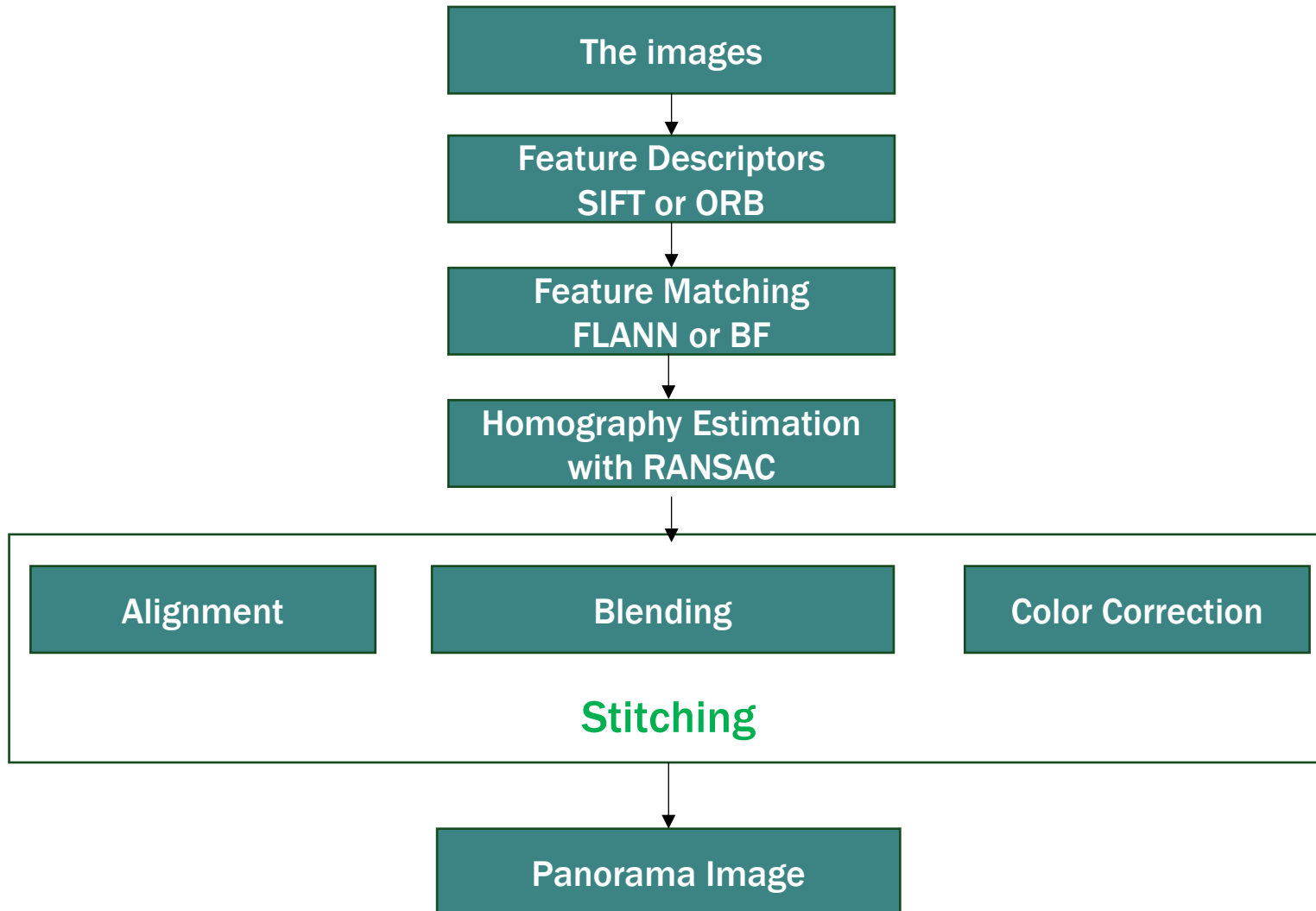
Based on Image Stitching

1. Detect keypoints and descriptors (SIFT, Speeded Up Robust Features-SURF, Oriented FAST and Rotated BRIEF-ORB)
2. Detect a set of matching points that is present in the images
3. Apply the RANSAC method to improve the matching process detection
4. Apply perspective transformation on one image using the other image as a reference frame
5. Stitch images together.



From: <https://pyimagesearch.com/2018/12/17/image-stitching-with-opencv-and-python/>

PANORAMA WITH TWO IMAGES



STITCHING TWO IMAGES

- Estimation homography matrix between two images (RANSAC for SIFT/ORB keypoints of them).
- Applying perspective transformation (transform an image from one projection angle to a completely different one).

```
1 def warpImages(img1, img2, H):
2     # H is homography matrix
3     rows1, cols1 = img1.shape[:2]
4     rows2, cols2 = img2.shape[:2]
5     listPoints1 = np.float32([[0,0], [0, rows1], [cols1, rows1], [cols1, 0]]).reshape(-1, 1, 2)
6     tempPoints = np.float32([[0,0], [0, rows2], [cols2, rows2], [cols2, 0]]).reshape(-1, 1, 2)
7     # When we have established a homography we need to warp perspective
8     # Change field of view
9     listPoints2 = cv2.perspectiveTransform(tempPoints, H)
10    listPoints = np.concatenate((listPoints1, listPoints2), axis=0)
11
12    [xMin, yMin] = np.int32(listPoints.min(axis=0).ravel() - 0.5)
13    [xMax, yMax] = np.int32(listPoints.max(axis=0).ravel() + 0.5)
14
15    translationDist = [-xMin, -yMin]
16
17    HTranslation = np.array([[1, 0, translationDist[0]], [0, 1, translationDist[1]], [0, 0, 1]])
18    output_img = cv2.warpPerspective(img2, HTranslation.dot(H), (xMax-xMin, yMax-yMin))
19    output_img[translationDist[1]:rows1+translationDist[1], translationDist[0]:cols1+translationDist[0]] = img1
20
21    return output_img
```

```
1 def getTransform(src, dst, method='affine'):
2     pts1, pts2 = featureMatching(src, dst)
3     srcPts = np.float32(pts1).reshape(-1,1,2)
4     dstPts = np.float32(pts2).reshape(-1,1,2)
5
6     if method == 'affine':
7         H, mask = cv2.estimateAffine2D(srcPts, dstPts, cv2.RANSAC,
8                                         ransacReprojThreshold=5.0)
9     if method == 'homography':
10        H, mask = cv2.findHomography(srcPts, dstPts, cv2.RANSAC, 5.0)
11        matchesMask = mask.ravel().tolist()
12    return (H, pts1, pts2, mask)
```

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- `cv2.findHomography(srcPts, dstPts, cv2.RANSAC, ransacReprojThreshold=5)`
- `cv2.warpPerspective(img, H, maxWid, maxHei, flags=cv2.INTER_BICUBIC)`

STITCHING MULTIPLE IMAGES

Multiple images stitching available in random order

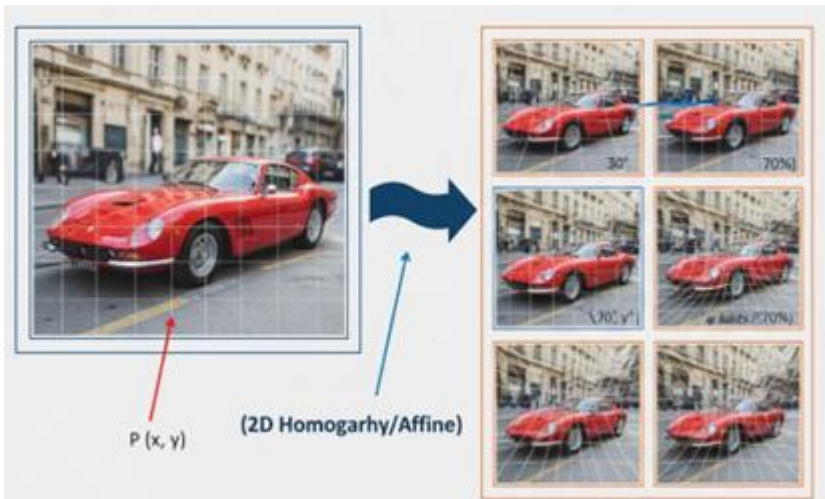
1. Performing some preprocessing steps, such as resizing and grayscale conversion, to enhance the results.
2. Compute all pairwise homographies H_{01} , H_{02} , H_{03} , H_{12} , H_{13} , ..., where H_{01} warp I_0 into I_1 .
3. Select one anchor image e.g. I_1 which position will remain fixed.
4. Find image that better align with I_1 based on **maximum number of consistent matches**, e.g. I_3 .
5. Now we have a new stitched image, and we can features in it and match them again in other frames and so on.

Multiple images stitching available in left-right order

1. Performing some preprocessing steps, such as resizing and grayscale conversion, to enhance the results.
2. **Left Stichting** for images in left half of the image list to get the *leftPano*.
3. **Right Stichting** for images in right half of the image list to get the *rightPano*.
4. Stitching leftPano to rightPano if rightPano's width is greater than leftPano's width and vice versa.s

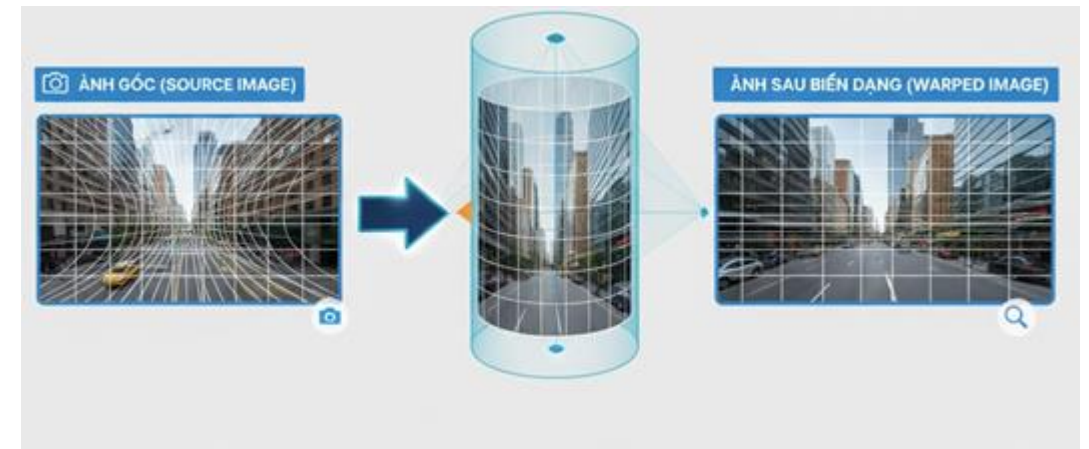
TYPES OF WARPING

Planar: This is the simplest type of warping, commonly used to perform basic geometric transformations such as translation, rotation, scaling, or perspective on a 2D plane. Pixels move in 2D space according to a linear or affine transformation. `.(cv2.warpPerspective)`



Spherical: Spherical warping is a technique for projecting a 2D image onto the surface of an imaginary sphere. This is the most extreme type of warping, often used to create full 360-degree panoramas from ultra-wide-angle photos.

Cylindrical: Cylindrical warping is often used to "straighten" an image taken with a wide-angle lens or to create panoramas. It projects pixels from a 2D plane onto the surface of an imaginary cylinder.



PANORAMA TWO IMAGES USING OPENCV'S STITCHER

```
[18] 1 def imagesSticher(images):
2     # initialize OpenCV's image sticher object and then perform the image
3     # stitching
4     print("[INFO] stitching images...")
5     stitcher = cv2.createStitcher() if imutils.is_cv3() else cv2.Stitcher_create()
6     (status, stitched) = stitcher.stitch(images)
7     # if the status is '0', then OpenCV successfully performed image
8     # stitching
9     return stitched, status

1 imagePaths = glob.glob('/content/drive/My Drive/computerVision/data/panorama/*.jpg')
2 images = []
3 for name in imagePaths:
4     image = cv2.imread(name)
5     images.append(image)
6 # image stitched
7 stitched, status = imagesSticher(images)
8 from matplotlib.pyplot import imshow
9 import matplotlib.pyplot as plt
10 if status == 0:
11     fig = plt.figure(figsize=(10, 10))
12     for i in range(1, len(images)+1):
13         fig.add_subplot(len(images)+1, 1, i)
14         plt.imshow(images[i-1])
15         plt.title("Image %i" %i)
16     # write the output stitched image to disk
17     # display the output stitched image to our screen
18     fig.add_subplot(len(images)+1, 1, len(images)+1)
19     plt.imshow(stitched)
20     plt.title("Stitched")
21     cv2.waitKey(0)
22     # otherwise the stitching failed, likely due to not enough keypoints)
23     # being detected
24 else:
25     print("[INFO] image stitching failed ({}).format(status))
```

